

Introdução ao TypeScript

Material de apoio — Programação Web | Prof. Pedro

1. O que é TypeScript?

TypeScript é um superset do JavaScript desenvolvido pela Microsoft que adiciona tipagem estática opcional à linguagem. Todo código JavaScript válido também é código TypeScript válido — a diferença é que o TypeScript permite declarar tipos para variáveis, parâmetros de função e retornos, o que ajuda a encontrar erros antes mesmo de rodar o programa, ainda durante a escrita do código no editor.

O código TypeScript não roda diretamente no navegador ou no Node.js: ele precisa ser compilado (transpilado) para JavaScript puro através do compilador oficial (tsc) ou de ferramentas como Vite, Babel ou esbuild.

2. Tipos básicos

TypeScript oferece tipos primitivos parecidos com os do JavaScript, mas agora declarados explicitamente:

```
let nome: string = "Davi";
let idade: number = 20;
let ativo: boolean = true;
let lista: number[] = [1, 2, 3];
let tupla: [string, number] = ["idade", 20];
```

Existe também o tipo **any**, que desativa a checagem de tipos para aquela variável — deve ser evitado sempre que possível, pois anula a principal vantagem do TypeScript. Uma alternativa mais segura é o tipo **unknown**, que exige uma verificação antes de a variável ser usada.

3. Interfaces

Interfaces descrevem o formato (shape) de um objeto — quais propriedades ele tem e de que tipo. São muito usadas para tipar dados vindos de APIs ou props de componentes.

```
interface Usuario {
  nome: string;
  idade: number;
  email?: string; // o "?" torna a propriedade opcional
}

function saudacao(usuario: Usuario): string {
  return `Olá, ${usuario.nome}!`;
}
```

Se um objeto passado para a função não tiver todas as propriedades obrigatórias da interface (nome e idade, nesse exemplo), o TypeScript aponta erro de compilação antes mesmo do código rodar.

4. Type Aliases vs Interfaces

Além de interfaces, é possível criar tipos personalizados com a palavra-chave **type**. A diferença principal é que type aliases podem representar uniões de tipos, o que interfaces não conseguem fazer sozinhas.

```
type Status = "pendente" | "aprovado" | "reprovado";

type Ponto = {
  x: number;
  y: number;
};
```

Uma regra prática comum: use **interface** quando estiver descrevendo objetos que podem ser estendidos (herança), e **type** quando precisar de uniões, interseções ou tipos primitivos combinados.

5. Enums

Enums permitem definir um conjunto nomeado de valores constantes relacionados, úteis para representar opções fixas como dias da semana ou status de um pedido.

```
enum StatusPedido {
  Pendente,
  Enviado,
  Entregue,
  Cancelado,
}

let statusAtual: StatusPedido = StatusPedido.Enviado;
```

Por padrão, cada valor do enum recebe um número inteiro começando em 0 (Pendente = 0, Enviado = 1, e assim por diante), mas é possível atribuir valores manualmente, inclusive strings.

6. Generics

Generics permitem criar funções, classes e interfaces reutilizáveis que funcionam com múltiplos tipos, mantendo a segurança de tipagem — em vez de usar **any** (que perde a checagem) ou duplicar código para cada tipo.

```
function identidade<T>(valor: T): T {
  return valor;
}

let numero = identidade<number>(10);
let texto = identidade<string>("oi");
```

Nesse exemplo, T é um parâmetro de tipo genérico: ele é substituído pelo tipo real no momento em que a função é chamada, sem que seja necessário escrever uma versão da função para cada tipo diferente.

7. Classes e modificadores de acesso

TypeScript adiciona aos modificadores de acesso das classes (recurso comum em linguagens orientadas a objeto) que controlam onde uma propriedade ou método pode ser acessado:

- **public**: acessível de qualquer lugar (padrão se nada for declarado)
- **private**: acessível apenas dentro da própria classe
- **protected**: acessível na própria classe e em subclasses

```
class ContaBancaria {  
    private saldo: number = 0;  
  
    depositar(valor: number): void {  
        this.saldo += valor;  
    }  
  
    consultarSaldo(): number {  
        return this.saldo;  
    }  
}
```

Como **saldo** é privado, ele não pode ser alterado diretamente de fora da classe (ex: **conta.saldo = 1000** geraria erro de compilação) — só através dos métodos públicos definidos, como **depositar()**. Isso é a base do encapsulamento em orientação a objetos.

8. Erros comuns de iniciantes

- Usar **any** em excesso, perdendo a principal vantagem da tipagem estática.
- Esquecer de instalar os tipos de bibliotecas JavaScript (pacotes `@types/...`) ao usar libs externas.
- Confundir **interface** com **type** e não saber quando usar cada um.
- Não configurar o `tsconfig.json` corretamente, deixando o modo strict desativado (o que permite muitos erros silenciosos passarem despercebidos).

Fim do material. Este conteúdo cobre os fundamentos de TypeScript: tipagem, interfaces, type aliases, enums, generics e classes com modificadores de acesso.